

CONTENTS OF THE CHAPTER

- *Generic programming: generic classes, creating objects from generic classes, generic methods, type limits.*
- *Collections framework: lists, collections, queues, maps.*

*Generic programming is an important technique, suitable for large-scale projects that involve similar objects and data handling. Generic programming reduces the number of lines of code needed, saving time and maintenance costs later on. Generic programming is a technique that is widely used in the Java Collections Framework, making data manipulation easier. This chapter on generic programming also introduces powerful Collection Frameworks in Java, including Lists, Sets, **and** Maps.*

6.1. INTRODUCTION

There are situations where methods and classes are independent of the data type they operate on. For example, an algorithm for finding an element in an array might handle a string array, an integer array, or a custom class. In this case, it doesn't matter what the array stores, while the algorithm itself is the same.

To make the code work with different data types, we can use two methods:

- Use inheritance.
- Generic programming.

The first method, using inheritance to create code that can work with many different object types, is a **classic programming approach**. However, this method is no longer widely used today.

Create a **MyList** class with the following code:

```
public class MyList {  
    private Object[] elementData;  
    private int size = 0;
```

```

MyList() {
    elementData = new Object[10];
}

public Object get(int i) {
    return elementData[i];
}

public void add(Object o) {
    elementData[size] = o;
    size++;
}
}

```

Write a main method using **MyList**:

```

public static void main(String[] args) {
    MyList files = new MyList();
    files.add("a.txt");

    String filename = (String) files.get(0);

    files.add(new File("a.txt"));
    files.add("a.txt");
}

```

There are two problems with using the Object type in this approach: firstly, **type casting is required** every time the get() method returns a value.

```

MyList files = new MyList();
String filename = (String) files.get(0);

```

The second problem is that the add method **does not perform type checking**; for example, in the case below, the function's parameter could be a File object or a string:

```

files.add(new File("..."));
files.add("...");

```

6.2. GENERIC PROGRAMMING

6.2.1. Generic Class

A generic class is a class that works with generic data types (types that don't need to be defined beforehand).

Use the following syntax to declare a generic class:

```
class ClassName<Type 1, Type 2, Type 3 ...>{  
  
}
```

In the following example, the **GenericClass** has a single type parameter named **T**. Assuming that type **T** is “something” and writing the class body that will work with this data type:

```
class GenericClass<T> {  
    private T t;  
  
    public GenericClass(T t) {  
        this.t = t;  
    }  
  
    public T get() {  
        return t;  
    }  
  
    public T set(T t) {  
        this.t = t;  
        return this.t;  
    }  
}
```

Once declared, a **type parameter** can be used inside the class body like a regular type. The example above uses the type parameter **T** as: type for a field, constructor argument type, method argument type, or return type.

The most commonly used type parameter names are:

- **T** – Generic type
- **E** – Element (widely used by collections)

- **K** – Key
- **V** – Value
- **N** – The number
- **S, U, V...** – Other types.

There is a naming convention that restricts the choice of type parameter names to single uppercase letters. Without this convention, it would be very difficult to distinguish between type variables and regular class names.

6.2.2. Creating objects from a Generic Class

To create an object from a generic class, you need to specify a “**type argument**”:

```
GenericClass<Integer> obj1 = new GenericClass<>(10);  
GenericClass<String> obj2 = new GenericClass<>("abc");
```

It is important to note that the type argument must be a **reference type**. **Primitive types** such as **int** or **double** cannot be used as type arguments. The **angle brackets** `<>` are informally known as the diamond operator.

Sometimes, declaring a variable with a generic type can be verbose and difficult to read. Starting with Java 10, it is possible to write `var` instead of a specific type to force automatic type inference based on the assigned value type.

```
var obj3 = new GenericClass<>("abc");
```

After creating an object with the specified type argument, you can call the methods of the class that takes or returns type parameters:

```
Integer number = obj1.get(); // 10  
String string = obj2.get(); // "abc"  
  
System.out.println(obj1.set(20));  
System.out.println(obj2.set("def"));
```

If a class has multiple type parameters, all of them must be specified when creating objects:

```
GenericClass<Type1, Type2, ..., TypeN> obj = new  
GenericClass<>(...);
```

6.3. Generic method

A **generic method** can be inside a generic class, or a regular class. For example, write the main method in a regular class and declare three arrays:

```
public static void main(String[] args) {  
    Double[] doubles = {1.2, 3.5, 2.5, 3.0, 6.0, 2.2};  
    Integer[] integers = {1, 0, -4, 3, 2, 3, 4, -1, 6, 2, 4};  
    String[] names = {"Peter", "David", "Harry"};  
}
```

Create a generic method called `getMiddle`:

```
public static <E> E getMiddle(E[] a) {  
    return a[a.length / 2];  
}
```

Use the generic method as follows:

```
public static void main(String[] args) {  
    ...  
    System.out.println(getMiddle(doubles));  
    System.out.println(getMiddle(integers));  
    System.out.println(getMiddle(names));  
}
```

Check the output displayed on the screen:

```
3.0  
3  
David
```

6.4. TYPE LIMITS

Use the syntax **<T extends A>** to restrict T to a subclass of A. For example:

```
public static <T extends Number> T getSum(T[] a) {
    Double sum = 0.0;
    for (T num : a) {
        sum = sum + num.doubleValue();
    }
    return (T) sum;
}
```

Use the method above:

```
public static void main(String[] args) {
    Double[] doubles = {1.2, 3.5, 2.5, 3.0, 6.0, 2.2};
    Integer[] integers = {1, 0, -4, 3, 2, 3, 4, -1, 6, 2, 4};
    String[] names = {"Peter", "David", "Harry"};

    System.out.println(getSum(doubles));
    System.out.println(getSum(integers));
}
```

Run the program and check the results displayed on the screen:

```
18.4
20.0
```

Use the syntax **<T extends B & C>** to restrict type T to be an implementation of interfaces B and C:

```
<T extends Comparable & Serializable>
```

6.5. COLLECTIONS FRAMEWORK

6.5.1. The concept of collections

A **collection** is an object used to contain or group other objects (usually of the same type).

- A deck of cards.
- An email folder.

- A phone directory.



Figure 6-1. Illustrative examples of sets.

6.5.2. Collections framework

Java provides **the Collections Framework**, which includes classes and interfaces for commonly reused data structures such as:

- List
- Set
- Queue
- Map
- ...

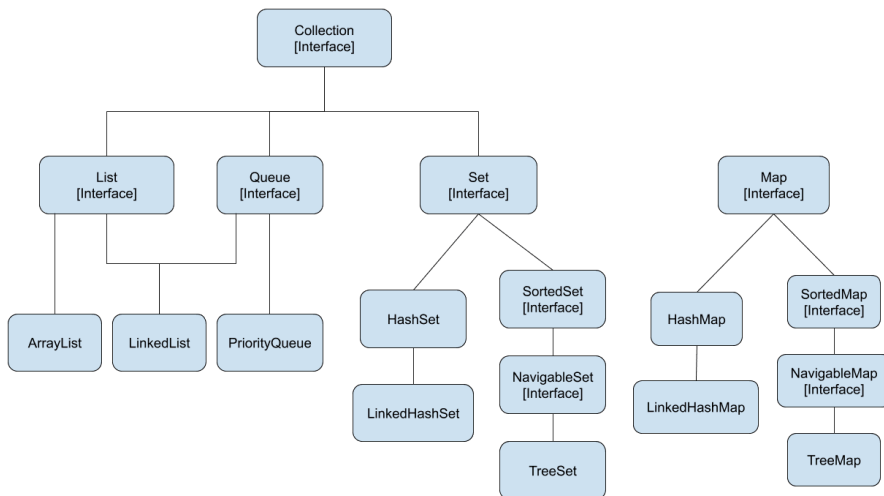


Figure 6-2. Main layers of the Collections framework

The **Collection<E>** interface represents an abstract collection, a container for objects of the same type. It provides several methods common to all other collection types.

The interfaces **List<E>**, **Set<E>**, **Queue<E>**, **SortedSet<E>**, and **Deque<E>** represent different types of collections. It is not possible to instantiate them directly. For example, the **ArrayList** class, which represents a resizable array, is the primary representation of the **List<E>** interface. Other interfaces, as well as their implementations, will be considered in later topics.

Another core interface is **Map<K,V>**, which represents a map for storing key-value pairs where **K** is the key type and **V** is the type of value stored. In the real world, a typical example of a map is a phone directory where the keys are names and the values are their phone numbers.

6.6. LIST

6.6.1. What is a List?

Lists are the closest type to arrays, except their size can be dynamically changed while arrays have a limited size. Furthermore, lists

offer more advanced behavior than arrays.

The List interface provides the following methods:

- **E set(int index, E element)** replaces the element at the specified position in this list with the specified element and returns the replaced element;
- **E get(int index)** returns the element at the specified position in the list;
- **int indexOf(Object obj)** returns the index of the first occurrence of the element in the list or **-1** if this index does not exist;
- **int lastIndexOf(Object obj)** returns the index of the last occurrence of the element in the list or **-1** if such an element does not exist;
- **List<E> subList(int fromIndex, int toIndex)** returns a sublist of this list from **fromIndex** (includes) to **toIndex** (excludes).

It's not possible to create an instance of a **List**, but you can create an instance of one of its implementations: **ArrayList**, **LinkedList**, or an immutable list, and then use it through the **List interface**. Programming to the List interface rather than to a specific implementation is considered good practice.

The simplest way to create a list is to call the 'of' method of **List**.

```
List<String> emptyList = List.of(); // 0 elements
List<String> names = List.of("Larry", "Kenny", "Sabrina"); //
3 elements
List<Integer> numbers = List.of(0, 1, 1, 2, 3, 5, 8, 13); // 8
elements
```

The “of” method returns an **immutable list** containing all the passed elements or an empty list. This method is convenient for creating list constants or checking some code.

```
List<String> daysOfWeek = List.of("Monday", "Tuesday",
"Wednesday", "Thursday", "Friday", "Saturday", "Sunday");
```

```
System.out.println(daysOfWeek.size()); // 7
System.out.println(daysOfWeek.get(1)); // Tuesday
System.out.println(daysOfWeek.indexOf("Sunday")); // 6

List<String> weekDays = daysOfWeek.subList(0, 5);
System.out.println(weekDays); // [Monday, Tuesday, Wednesday,
Thursday, Friday]
```

Because it is **immutable**, only methods that do not change the elements in the list will work. Other methods will throw an exception.

```
daysOfWeek.set(0, "Funday"); // throws
UnsupportedOperationException
daysOfWeek.add("Holiday"); // throws
UnsupportedOperationException
```

Another way to create it is to use the **asList** method.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
```

6.6.2. Traversing the list

There are **three ways** to iterate through a list. Consider the following list as an example:

```
List<String> names = List.of("Larry", "Kenny", "Sabrina");
```

Use a “for i” loop:

```
for (int i = 0; i < names.size(); i += 2) {
    System.out.println(names.get(i));
}
```

6.6.3. Using the “for-each ” loop

```
for (String name : names) {
    System.out.println(name);
}
```

6.6.4. Using the forEach method

```
names.forEach(System.out::println);
```

6.6.5. ArrayList

When **mutable lists** are needed, one of two commonly used mutable implementations of the List interface can be used.

```
List<Integer> numbers = new ArrayList<>();
numbers.add(15);
numbers.add(10);
numbers.add(20);
System.out.println(numbers); // [15, 10, 20]
numbers.set(0, 30); // no exceptions here
System.out.println(numbers); // [30, 10, 20]
```

The logic of an **ArrayList** is very simple: because it is an array inside, the **get(int index)** and **set(int index, E element)** operations will execute quickly. In other words, the time complexity to access by an index is $O(1)$.

the **add(E e)** operation, a new element will be added to the end of the ArrayList. This will also be fast and require a fixed amount of time to execute, so the complexity will be the same: $O(1)$.

However, if you want to insert an element into an **ArrayList** using **add(int index, E element)**, the process is different. Specifically, to place the new element at a specific index, the **ArrayList** needs to move all subsequent elements.

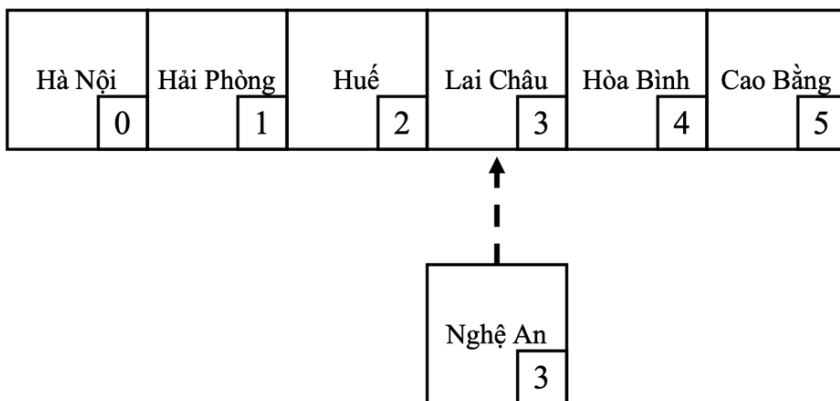


Figure 6-3. ArrayList stores elements in array form.

In this example, after insertion, the "Array" element will have a new

index of 4 and the list will have a new index of 5. If there are many elements after it, this operation will take quite a long time with a time complexity of $O(n)$.

When adding a new element to a full `ArrayList`, a larger new array is first created. Then, all existing elements are copied into this new array. Only then is the new element added. Therefore, the time complexity is $O(n)$.

An **ArrayList** is a resizable array, so as it fills up to its original size, it becomes larger, and this process repeats itself.

The final operation is `remove(int index)`. When you want to remove an element, all subsequent elements must be moved.

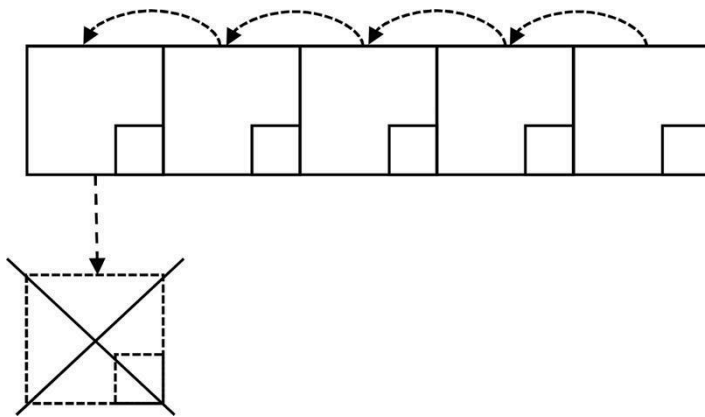


Figure 6-4. Shifting elements in an ArrayList when deleting an element.

Because it involves moving all subsequent elements, this operation would not be very efficient for a long list, with a time complexity of $O(n)$.

6.6.6. LinkedList

Another implementation of the **List interface** is the **LinkedList class**. It represents a doubly linked list based on connected nodes. All indexing operations on the list will traverse the list from the beginning or from the end, whichever is closest to the specified index.

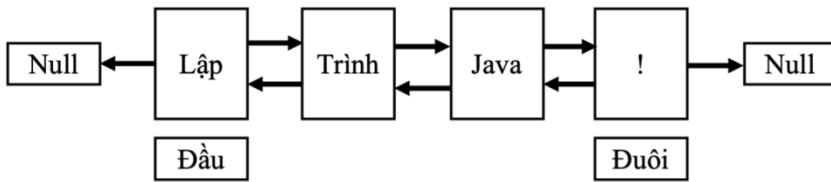


Figure 6-5. LinkedList organizes data as a linked list.

LinkedLists don't support fast index access, but they can access an element by its index. To do this, LinkedList decides which element is closer to that index: the head or the tail. Then, starting from the head or the tail, LinkedList iterates through all the elements before reaching the required element.

To get the element at index 1, the LinkedList will start from the beginning. In this case, the LinkedList will only need to traverse one link. But if there is a long LinkedList and the required index is in the middle, the `get(int index)` and `set(int index, E element)` operations will take a lot of time. So in a worst-case scenario, the time complexity will be $O(n)$.

```
List<Integer> numbers = new LinkedList<>();
numbers.add(10);
numbers.add(20);
numbers.add(30);
System.out.println(numbers); // [10, 20, 30]
```

Adding an element to the end of a **LinkedList** is a quick operation.

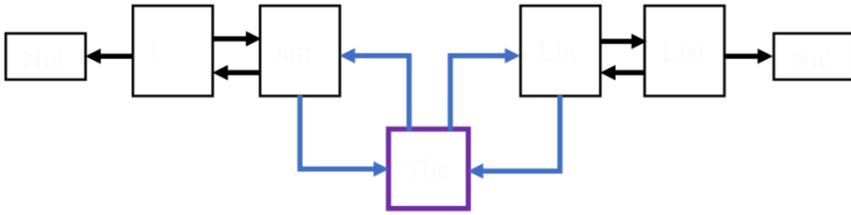


Figure 6-6. Adding an element to the end of a list.

Therefore, it always takes a fixed amount of time and the complexity of **add(E e)** is $O(1)$.

Besides the **add(E e) method**, **LinkedList** has two other methods: **addFirst(E e)** and **addLast(E e)**. These methods add elements to the head and tail of the **LinkedList**, respectively. Their internal mechanism is exactly the same as the **add(E e) method**, and their time complexity is also $O(1)$. The **addLast(E e) method** is equivalent to the **add(E e) method**. The difference is that the **add(E e) method** returns a **boolean** and the **addLast(E e) method** returns **void**.

To add an element, the **LinkedList** needs to reach the required position and then only change the links of the new neighboring elements.

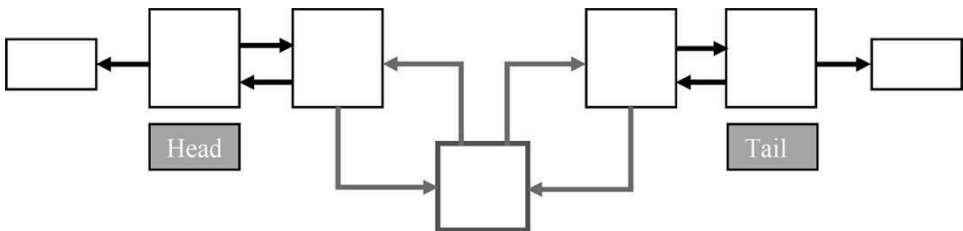


Figure 6-7. Adding an element to the middle of a list.

Connect the new element to the previous and next elements, and the insertion is complete. There's no need to move elements as in an **ArrayList**.

If you add new elements near the beginning or end, the time complexity will be $O(1)$. In most situations, that's a fast operation. But if the

list is very long, accessing the middle element won't be as fast. So, in a bad situation, the time complexity will be $O(n)$.

`remove()` method has a similar mechanism to the `add` method. To remove an element from the middle of a `LinkedList`, the time complexity will be $O(n)$.

`LinkedList` has two other useful methods: `removeFirst()` and `removeLast()`. Using these two methods, `LinkedList` doesn't need to iterate through the elements. It simply removes the first or last element and changes the beginning or end accordingly. The time complexity for both operations is $O(1)$.

6.6.7. Compare `ArrayList` vs `LinkedList`

The difference between an `ArrayList` and a `LinkedList` lies in how they store and organize their elements:

- An `ArrayList` is a resizable array of objects where every element has an index.
- A `LinkedList` is a doubly linked list based on connected nodes. **The `LinkedList` stores its head and tail.**

Comparison table:

Table 6-1. Comparison of the speed of `ArrayList` and `LinkedList`

	<code>ArrayList</code>	<code>LinkedList</code>
<code>get(int index)</code>	$O(1)$	$O(n)$
<code>set(int index, E element)</code>	$O(1)$	$O(n)$
<code>add(E e)</code>	$O(1)$	$O(1)$
<code>add(int index, E element)</code>	$O(1) + O(n)$	$O(n) + O(1)$
<code>remove(int index)</code>	$O(1) + O(n)$	$O(n) + O(1)$

`ArrayLists` are more common, but in reality, it all depends on the task at hand. For any operation, it's impossible to say in advance which class will

perform faster. It depends on the size of the list and which part of the list you're working with. The most important thing to understand is how each type of operation works.

6.7. SET

6.7.1. The concept of Set

A **set** is a collection of unique elements. A set differs from a list or an array because elements cannot be accessed by index. A Set should be used when uniqueness of elements must be enforced.

Set inherits all methods from the `Collection<E>` interface. The most widely used methods include **contains**, **add**, **addAll**, **remove**, **removeAll**, and **size**.

add and **addAll** methods only add elements to the collection if those elements are not already in **the Set**. A **Set** always contains only unique elements. One method worth noting when discussing **the Set<E> interface**, as it is often used with collections, is **retainAll(Collection<E> coll)**.

It only retains the elements already contained in the specified collection. To begin using a collection, one needs to initialize one of its implementations: **HashSet**, **TreeSet**, and **LinkedHashSet**. These are mutable collections and they use different rules to sort elements and have several additional methods. They are also optimized for different types of operations.

The simplest way to create a collection is to call the **`of()` method of the `Set` interface**. It returns an immutable collection containing all the elements passed in, or an empty collection. Using this **`of` method** is convenient when creating constants or testing some code.

```
Set<String> emptySet = Set.of();  
Set<String> persons = Set.of("Larry", "Kenny", "Sabrina");  
Set<Integer> numbers = Set.of(100, 200, 300, 400);
```

The iteration order of an immutable set is not guaranteed:

```
System.out.println(emptySet); // []
```

```
System.out.println(people); // [Kenny, Larry, Sabrina] or
another order
System.out.println(numbers); // [400, 200, 300, 100] or another
order
```

One of the most widely used set operations is checking whether a set contains an element. Here's an example:

```
System.out.println(emptySet.contains("hello")); // false
System.out.println(people.contains("Sabrina")); // true
System.out.println(people.contains("John")); // false
System.out.println(numbers.contains(300)); // true
```

For immutable collections, only the `contains`, `size`, and `isEmpty` methods can be called. All other methods will throw an `UnsupportedOperationException` for attempting to modify the collection. If you want to add or remove elements, you can use one of the `HashSet`, `TreeSet`, or `LinkedHashSet` methods.

Next, we will look at three main possible implementations of the `Set` interface.

6.7.2. HashSet

`HashSet` class represents a collection backed by a hash table. It uses the hash codes of the elements to store them efficiently.

The following example illustrates creating a `HashSet` and adding countries to it. The output does not contain duplicate elements.

```
Set<String> countries = new HashSet<>();

countries.add("India");
countries.add("Japan");
countries.add("Switzerland");
countries.add("Japan");
countries.add("Brazil");

System.out.println(countries); // [Japan, Brazil, Switzerland,
India]
```

```
System.out.println(countries.contains("Switzerland")); // true
```

Although technically the order of a **HashSet** is somewhat determined by the hash code, **a HashSet does not guarantee iteration order.**

HashSets have $O(1)$ complexity for basic operations (**add**, **remove**, and **contains**), assuming the hashing method distributes the elements correctly among the buckets.

6.7.3. TreeSet

The **TreeSet** class represents a collection of sorted elements. TreeSets **implement** the **SortedSet interface**, which extends the base **Set interface**. The **SortedSet** interface provides several new methods related to comparing elements:

- `Comparator<? super E> comparator()` returns a comparator used to sort the elements in the set, or **null** if the set uses the natural order of its elements;
- `SortedSet<E> headSet(E toElement)` returns a subset containing elements smaller than **toElement** ;
- `SortedSet<E> tailSet(E fromElement)` returns a subset containing elements greater than or equal to **fromElement** ;
- `SortedSet<E> subSet(E fromElement, E toElement)` returns a subset containing elements within the range **fromElement** (include) **toElement** (exclude);
- **E first()** returns the first (lowest) element in the set;
- **E last()** returns the last (highest) element in the set.

The following example illustrates some of these methods:

```
SortedSet<Integer> sortedSet = new TreeSet<>();

sortedSet.add(10);
sortedSet.add(15);
sortedSet.add(13);
```

```
sortedSet.add(21);
sortedSet.add(17);

System.out.println(sortedSet); // [10, 13, 15, 17, 21]

System.out.println(sortedSet.headSet(15)); // [10, 13]
System.out.println(sortedSet.tailSet(15)); // [15, 17, 21]

System.out.println(sortedSet.subSet(13,17)); // [13, 15]

System.out.println(sortedSet.first()); // minimum is 10
System.out.println(sortedSet.last()); // maximum is 21
```

HashSet is much faster than **TreeSet** : the time is constant relative to the log-time for most operations. If you need to use operations from the **SortedSet** interface, use **TreeSet** ; otherwise, **HashSet** is a better choice.

6.8. MAP

A **map** is a collection of key-value pairs. The keys are always unique, while the values can be repeated.

Let's look at a phone book as an example:

```
Keys : Values
-----
Bob: +1-202-555-0118
James: +1-202-555-0220
Katy: +1-202-555-0175
```

Maps have some similarities to sets and lists:

- The keys of a Map form a Set, but each key has an associated value;
- The keys of a map are similar to the indices of an array, but the keys can be of any type including integers, strings, etc.

The interface declares many methods for working with **Maps**. Some

methods are similar to Collection methods, while others are Map-specific:

Methods similar to collections:

- **int size()** returns the number of elements in the Map;
- **boolean isEmpty()** returns **true** if the Map does not contain elements and **false** otherwise.
- **void clear()** removes all elements from the Map.

Handling keys and values:

- **V put(K key, V value)** It associates **the value** with the specified **key** and returns the value previously associated with the **key**, or null.
- **V get(Object key)** returns the value associated with the key or **null** if not found;
- **V remove(Object key)** removes the mapping for a key from the Map;
- **boolean containsKey(Object key)** returns true if the Map contains the key;
- **boolean containsValue(Object value)** returns true if the Map contains the value.

Methods that return different sets:

- **Set<K> keySet()** returns a **Set** of keys contained in this Map;
- **Collection<V> values()** returns a **Collection** that shows the values in this Map;
- **Set<Map.Entry<K, V>> entrySet()** returns a **Set of** entries (associations) **that** are in this Map.

Advanced method:

- The **putIfAbsent(K key, V value)** statement sets a pair if the specified key is not already associated with a value (or is mapped to null) and returns null; otherwise, it returns the current value.

- **V getOrDefault(Object key, V defaultValue)** returns the value that the specified key is mapped to, or **defaultValue** if this Map does not contain a mapping for the key.

The simplest way to create a Map is to call the **`of` method of the Map** interface. This method leaves the arguments empty or an even number in the format **key1, value1, key2, value2,...** and returns an immutable Map.

```
Map<String, String> emptyMap = Map.of();
Map<String, String> friendPhones = Map.of(
    "Bob", "+1-202-555-0118",
    "James", "+1-202-555-0220",
    "Katy", "+1-202-555-0175"
);
```

Now let's look at some operations that can be applied to immutable Maps using the example of **friendPhones**.

The size of a Map is equal to the number of pairs it contains.

```
System.out.println(emptyMap.size()); // 0
System.out.println(friendPhones.size()); // 3
```

You can retrieve values from a Map using its key:

```
String bobPhone = friendPhones.get("Bob"); // +1-202-555-0118
String alicePhone = friendPhones.get("Alice"); // null
String phone = friendPhones.getOrDefault("Alex", "Unknown
phone"); // Unknown phone
```

It is also possible to check if a map contains a specific key or value using the **`containsKey`** and **`containsValue`** methods.

Directly access the key set and collection of values from the map:

```
System.out.println(friendPhones.keySet()); // [James, Bob,
Katy]
System.out.println(friendPhones.values()); // [+1-202-555-0220,
+1-202-555-0118, +1-202-555-0175]
```

6.8.1. HashMap

HashMap class represents a Map backed by a hash table. This

implementation provides fixed-time complexity for the **get** and **put methods** (assuming the hash method distributes elements correctly among the buckets).

The following example illustrates a Map of products where the key is the product code and the value is the name.

```
Map<Integer, String> products = new HashMap<>();

products.put(1000, "Notebook");
products.put(2000, "Phone");
products.put(3000, "Keyboard");

System.out.println(products); // {2000=Phone, 1000=Notebook,
3000=Keyboard}

System.out.println(products.get(1000)); // Notebook

products.remove(1000);

System.out.println(products.get(1000)); // null

products.putIfAbsent(3000, "Mouse"); // it does not change the
current element

System.out.println(products.get(3000)); // Keyboard
```

6.8.2. TreeMap

TreeMap class represents a Map that provides us with a guarantee of the order of elements. It corresponds to the sorting order of keys defined according to their natural order (if they implement the Comparable interface) or implement a specific Comparator.

This class implements the **SortedMap interface**, which extends the base **Map interface**. It provides several new methods related to comparing keys:

- `Comparator<? super K> comparator()` returns a comparison

method used to sort the elements in the map, or **null** if the map uses the natural order of its keys;

- **E firstKey()** returns the first (lowest) key in the map;
- **E lastKey()** returns the last (highest) key in the map;
- **SortedMap<K, V> headMap(K toKey)** returns a submap containing elements with keys smaller than **toKey** ;
- **SortedMap<K, V> tailMap(K fromKey)** returns a submap containing elements whose key is greater than or equal to **fromKey** ;
- **SortedMap<K, V> subMap(K fromKey, E toKey)** returns a submap containing elements whose keys are in the range **fromKey** (inclusive) **toKey** (exclusive);

TreeMap object. This map contains events, each with a date (key) and a title (value).

LocalDate is a class that represents a date. Calling the **LocalDate.of(year, month, day)** method creates the specified date object with the given **year, month, and day**.

```
SortedMap<LocalDate, String> events = new TreeMap<>();

events.put(LocalDate.of(2017, 6, 6), "The Java Conference");
events.put(LocalDate.of(2017, 6, 7), "Another Java
Conference");
events.put(LocalDate.of(2017, 6, 8), "Discussion: career or
education?");
events.put(LocalDate.of(2017, 6, 9), "The modern art");
events.put(LocalDate.of(2017, 6, 10), "Coffee master class");

LocalDate fromInclusive = LocalDate.of(2017, 6, 8);
LocalDate toExclusive = LocalDate.of(2017, 6, 10);

System.out.println(events.subMap(fromInclusive, toExclusive));
```

Output:

```
{2017-06-08=Discussion: career or education?, 2017-06-09=The modern art}
```

It's important to note that **TreeMap** should be used when sorted ordering of keys is required, as this implementation is less efficient than **HashMap**.

6.8.3. Traversing a Map

Maps are typically navigated using keys or values.

```
Map<String, String> friendPhones = Map.of(
    "Bob", "+1-202-555-0118",
    "James", "+1-202-555-0220",
    "Katy", "+1-202-555-0175"
);

for (String name : friendPhones.keySet()) {
    System.out.println(name);
}

for (String phone : friendPhones.values()) {
    System.out.println(phone);
}
```

If you want to print a key and its associated value in the same iteration, you can use **entrySet()** and iterate over it.

```
for (var entry : friendPhones.entrySet()) {
    System.out.println(entry.getKey() + ": " +
entry.getValue());
}
```

This code prints all key-value pairs:

```
Bob: +1-202-555-0118
James: +1-202-555-0220
Katy: +1-202-555-0175
```

Similar behavior can be achieved by using a lambda expression with

two arguments:

```
friendPhones.forEach((name, phone) -> System.out.println(name +  
": " + phone));
```

6.9. ITERATOR

Iterators `<T>` are a common mechanism for iterating through sets regardless of their structure. **Iterators** allow removing elements from the underlying set, but this cannot be done using a for-each loop.

Some methods of the `Iterator<E>` interface:

- **boolean hasNext()** returns **true** if there are still elements and **false** if there are no elements.

Otherwise:

- **E next()** returns the next element in the loop;
- The **void remove()** method removes the last element returned by this iterator from the collection.

Typical usage involves three steps:

1. Check the collection for the next element.
2. Move to the next element.
3. Process the resulting element.

```
Set<Long> set = new TreeSet<>(); // sorted set  
set.add(10L);  
set.add(5L);  
set.add(18L);  
set.add(14L);  
set.add(9L);  
  
System.out.println(set); // [5, 9, 10, 14, 18]  
  
Iterator<Long> iter = set.iterator();  
while (iter.hasNext()) {
```

```
        Long current = iter.next();
        if (current < 10L) {
            iter.remove();
        }
    }

    System.out.println(set); // [10, 14, 18]
```

6.10. COLLECTIONS UTILITY CLASS

The Java Collections framework includes the **'Collections' utility class**, which contains several static methods for creating and processing collections. Programmers often forget about this class and rewrite existing methods. Clearly, it's better to remember this class and check if it contains the necessary operations to perform on a collection. Note, do not confuse the **'Collections' class with the 'Collections' interface**.

6.10.1. List Processing

Additionally, there are several methods for performing list-specific operations: sorting, reversing, rotating, and shuffling lists. For example:

```
var numbers = new ArrayList<>(List.of(1, 2, 3, 2, 3, 4));

Collections.sort(numbers);      // [1, 2, 2, 3, 3, 4]
Collections.reverse(numbers);   // [4, 3, 3, 2, 2, 1]
Collections.shuffle(numbers);   //

System.out.println(numbers);    // a result can be any: [4, 2, 3,
2, 3, 1]
```

rotate methods move elements in a list by a given distance.

```
List<Integer> numbers = new ArrayList<>(List.of(1, 2, 3, 2, 3,
4));

Collections.rotate(numbers, 1); // [4, 1, 2, 3, 2, 3]
Collections.rotate(numbers, 2); // [2, 3, 4, 1, 2, 3]
```

6.10.2. Calculations on the collection

There are several methods that can be applied to any collection because these methods take the **Collection interface as an argument**.

- **Frequency** counts the number of elements using a defined object;
- **min** and **max** find the minimum and maximum elements in the natural order of the elements;
- **A disjoint** method checks whether two collections have no elements in common.

This is an example of applying the methods listed.

```
List<Integer> numbers = List.of(1, 2, 3, 2, 3, 4);

System.out.println(Collections.frequency(numbers, 3)); // 2
System.out.println(Collections.min(numbers)); // 1
System.out.println(Collections.max(numbers)); // 4

System.out.println(Collections.disjoint(numbers, List.of(1,
2))); // false
System.out.println(Collections.disjoint(numbers, List.of(5,
6))); // true
```

min and **max** methods will throw a **NoSuchElementException**, but **frequency** will return 0.

6.10.3. Other Features

A complex and interesting example involving several modifications to an immutable set.

```
List<Integer> singletonList = Collections.singletonList(1);

Collections.sort(singletonList);      //
Collections.shuffle(singletonList);
exception
```

```
List<Integer> numbers = Collections.unmodifiableList(List.of(2, 1, 3));  
Collections.shuffle(numbers);
```

PRACTICE

Exercise 1. The Generic Class Pair

Create a Pair class with the following content:

```
public class Pair <T> {  
    private T first ;  
    private T second ;  
    public Pair (T first, T second) {  
        this.first = first ;  
        this.second = second ;  
    }  
    public T getFirst () {  
        return first ;  
    }  
    public T getSecond () {  
        return second ;  
    }  
    public void setFirst (T first) {  
        this.first = first ;  
    }  
    public void setSecond (T second) {  
        this.second = second ;  
    }  
}
```

Write a main method using this Pair class:

```
public static void main (String [] args) {  
    Pair <String>pairString = new Pair("Hello", "World");  
    Pair <Integer>pairInteger = new Pair(1, 3);  
    Pair <Double>pairDouble = new Pair(- 2.5, 1.3);  
  
    Pair pair = new Pair<>("Hello", "World");
```

```

pair = new Pair(1, 3);
pair = new Pair(- 2.5, 1.3);
    System.out.println(pairString.getSecond());
    System.out.println(pair.getFirst());
}

```

The result will be displayed on the screen:

```

World
-2.5

```

Further practice: Write a swap method to swap the first and second options.

Exercise 2. Type Limits

Write a method that returns the minimum and maximum values of the elements in an array of type T:

```

public static <T extends Comparable>Pair <T>minmax (T [] a) {
    if (a == null || a.length == 0) return null ;
    T min = a[0];
    T max = a[0];
    for (int i = 1 ; i < a.length ; i++) {
        if (min.compareTo(a[i])>0) min = a[i];
        if (max.compareTo(a[i]) <0) max = a[i];
    }
    return new Pair<>(min, max);
}

```

Use the method described above to find the maximum and minimum values:

```

public static void main (String [] args) {
    Integer [] numbers = {2, 3, 5, 2, 5};
    LocalDate [] birthdays =
    {
        LocalDate.of (1906, 12, 9),
        LocalDate.of (1815, 12, 10),
        LocalDate.of (1903, 12, 3),
    }
}

```

```

        LocalDate.of(1910, 6, 22)
    };

    Pair <Integer>pairMinMaxNum = minmax (numbers);
    System.out.println("Min:" + pairMinMaxNum.getFirst() + " - 
Max:" + pairMinMaxNum.getSecond());
    Pair <LocalDate>pairMinMaxDay = minmax (birthdays);
    System.out.println("Min:" + pairMinMaxDay.getFirst() + " - 
Max:" + pairMinMaxDay.getSecond());
}

```

The result will be displayed on the screen:

```
Min: 2 - Max: 5
```

```
Min:1815-12-10 - Max:1910-06-22
```

Exercise 3. ArrayList

Create a new list called arrayList:

```
ArrayList <String>list = new ArrayList<>();
```

Add five types of fruit to arrayList and print it out:

```

list.add ("apples");
list.add ("bananas");
list.add ("grapes");
list.add ("pears");
list.add ("black plums");
System.out.println("My array list currently includes: " + 
list);

```

Calling some methods on the list:

```

list.add(0, "watermelons");
System.out.println("My array list now includes: " + list);

list.add (3, "oranges");
System.out.println("My array list now includes: " + list);

System.out.println("'grapes' is an element in the list: " + 
list.contains("grapes"));

```

```

System.out.println("'apricots' is an element in the list: " +
list.contains("apricots"));

System.out.println("Element at index 2 is: " + list.get(2));

System.out.println("Element at index 1 is: " + list.get(1));

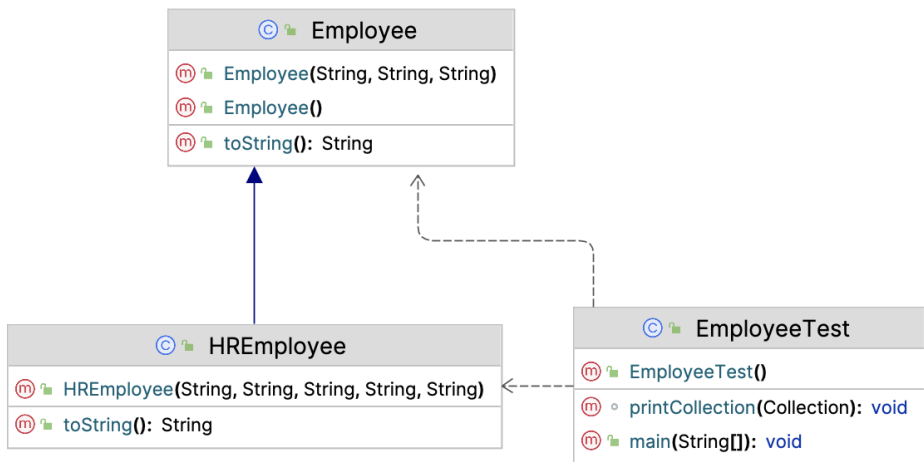
System.out.println("The size of our current array list is: " +
list.size());

```

Exercise (*) Write additional source code to:

- Check if "lemons" is in the list.
- Is the last element of the list "bananas"?
- Put "cherries" after "apples".
- Remove "grapes" if present.
- Convert the characters in the list to uppercase.

Exercise 4. Employee List



Create an Employee class:

```

public class Employee {
    private String employeeName ;

```

```

private String employeeNo ;
private String employeeDescription ;

public Employee () {

}

public Employee (String eName, String eNo, String
eDescription) {
    employeeName = eName;
    employeeNo = eNo;
    employeeDescription = eDescription;
}

public String toString () {
    return "Name: " + employeeName + " \t Number: " +
employeeNo + " \t Designation: " + employeeDescription ;
}
}

```

Create the HREmployee class:

```

public class HREmployee extends Employee {
    private String deptName ;
    private String deptCode ;

    public HREmployee (String name, String num, String desig,
String dept, String code) {
        super (name, num, desig);
        deptName = dept;
        deptCode = code;
    }

    public String toString () {
        return super.toString() + " \t Department Name: " +
deptName + " \t Department Code:" + deptCode ;
    }
}

```

Program running:

```
public class EmployeeTest {

    static void printCollection (Collection collect) {
        for (Object obj : collect)
            System.out.println(obj.toString());
    }

    public static void main (String [] args) {
        ArrayList <Employee>emp = new ArrayList<>(1);
        System.out.println("Employee Class Details: ");
        System.out.println("*****");
        emp.add(0, new Employee("Raphael John", "A001", "Team
Lead"));
        emp.add(1, new Employee("Jenny Samuel", "A002", "Project
Manager"));
        printCollection(emp);
        ArrayList <HREmployee>empHR = new ArrayList<>(1);
        System.out.println(" \n HREmployee Class Details: ");
        System.out.println("*****");
        empHR.add(0, new HREmployee("Angelina John", "A001",
"Project Manager", "Operations", "01"));
        empHR.add(1, new HREmployee("Susan Pitt", "A002", "Team
Leader", "Writing", "02"));
        printCollection (empHR);
    }
}
```

Exercise 5. Student List

Create a Student class:

```
public class Student {

    String StudentName ;
    String StudentID ;
    String description ;
    int Studentage ;
}
```

```

    public Student () {
    }

    public Student (String pName, String pID, String descr, int
age) {
        StudentName = pName;
        StudentID = pID;
        description = descr;
        Studentage = age;
    }

    public String toString () {
        return "Student Name: " + StudentName + " \t Student ID:
" + StudentID + " \t Description: " + description + " \t age: "
+ Studentage ;
    }
}

```

Create a PeopleList class:

```

public class PeopleList <T>{
    ArrayList <T> listPeople ;

    public PeopleList () {
        listPeople = new ArrayList<T>();
    }

    public void add (T obj) {
        listPeople.add (obj);
    }

    public void display () {
        for (T objPeople : listPeople) {
            System.out.println(objPeople.toString());
        }
    }
}

```

```

public int getSize () {
    return listPeople.size ();
}

public boolean checkEmpty () {
    return (listPeople.size() == 0);
}

public T grab () {
    if (!checkEmpty()) {
        return listPeople.remove(0);
    }
    else
    return null ;
}
}

```

Create a program to run:

```

public class Program {
    public static void main (String [] args) throws Exception {
        PeopleList <Student>objStudent = new
        PeopleList<Student>();
        objStudent.add(new Student("Pham Xuan Minh", "123",
"Class1", 22));
        objStudent.add (new Student("Duong Mai Huong", "124",
"Class2", 20));
        objStudent.add(new Student("Tran Minh Tuan", "125",
"Class1", 19));
        System.out.println("Is the list empty? " +
objStudent.checkEmpty());
        System.out.println("Size of the list: " +
objStudent.getSize());
        System.out.println("Student Details:");
        System.out.println("*****");
        objStudent.display();
    }
}

```

Implement the program:

```
Is the list empty? false
```

```
Size of the list: 3
```

```
Student Details:
```

```
*****
```

```
Student Name: Pham Xuan Minh Student ID: 123Description:  
Class1age: 22
```

```
Student Name: Duong Mai Huong Student ID:  
124Description: Class2age: 20
```

```
Student Name: Tran Minh Tuan Student ID: 125Description:  
Class1age: 19
```

Exercise : Modify the program so that students print out the list alphabetically (Names in ABC order).

Exercise 6. Linked List

Create an ExLinkedList class with a main method, which will create the LinkedList:

```
LinkedList <Integer>list = new LinkedList<>();
```

Add data to the list:

```
list.add (11);  
list.add (22);  
list.add (33);  
list.add (44);
```

Create an iterator to print the list:

```
System.out.print("Linked list data: ");  
Iterator iterator = list.iterator();  
while (iterator.hasNext())  
    System.out.print(iterator.next() + " ");  
System.out.println();
```

Check if the list is empty:

```
if (list.isEmpty ())  
    System.out.println("Linked list is empty");
```

```
else
```

```
System.out.println("Linked list size: " + list.size());
```

Add an element to the beginning of the list:

```
System.out.println("Adding data at 1st location: 55");
list.addFirst (55);
System.out.print("Now the list contains: ");
iterator = list.iterator ();
while (iterator.hasNext())
    System.out.print(iterator.next() + " ");
System.out.println();
System.out.println("Now the size of list: " + list.size());
```

Add an element to the end of the list:

```
System.out.println("Adding data at last location: 66");
list.addLast (66);
System.out.print("Now the list contains: ");
iterator = list.iterator ();
while (iterator.hasNext())
    System.out.print(iterator.next() + " ");
System.out.println();
System.out.println("Now the size of list: " + list.size());
```

Add the element at the 3rd position:

```
System.out.println("Adding data at 3rd location: 55");
list.add (2, 99);
System.out.print("Now the list contains: ");
iterator = list.iterator ();
while (iterator.hasNext())
    System.out.print(iterator.next() + " ");
System.out.println();
System.out.println("Now the size of list: " + list.size());
```

Print the elements at the beginning and end:

```
System.out.println("First data: " + list.getFirst());
System.out.println("Last data: " + list.getLast());
```

Remove the element at the end:

```
//Remove last
int last = list.removeLast();
System.out.println("Data removed from last location: " +
last);
System.out.print("Now the list contains: ");
iterator = list.iterator ();
//After removing data
while (iterator.hasNext())
    System.out.print(iterator.next()+ " ");
System.out.println();
System.out.println("Now the size of list: " + list.size());
```

Delete all elements:

```
//Remove all
list.clear();
if (list.isEmpty ())
    System.out.println("Linked list is empty");
else
    System.out.println("Linked list size: " + list.size());
```

Run the program and explain each output displayed on the screen.

Exercise 7. HashSet and TreeSet

Create an ExHashSet class with a main method, and within that, create a HashSet named s:

```
Set <String>s = new HashSet<>();
```

Add elements to the HashSet and print the result:

```
s.add("grapes");
s.add("bananas");
s.add("apples");
s.add("pears");
s.add("black plums");
System.out.println("My set currently includes: " + s);
s.add("watermelons");
System.out.println("My set now includes: " + s);
```

```
s.add("bananas");  
System.out.println("My set now includes: " + s);
```

Check if the set contains the elements **grapes** and **apricots** :

```
// Determines whether or not 'grapes' is in the s.  
System.out.println("'grapes' is an element in the set: " +  
s.contains("grapes"));  
// Determines whether or not 'apricots' is in the s.  
System.out.println("'apricots' is an element in the set: " +  
s.contains("apricots"));
```

Remove elements from a HashSet and display the size:

```
// Checks to see if s is empty or not.  
System.out.println("The set is currently empty: " +  
s.isEmpty());  
  
// Returns the current size of the array.  
System.out.println("The size of set is: " + s.size());
```

Run the program and check the results:

```
'grapes' is an element in the set: true  
'apricots' is an element in the set: false
```

Exercise (*): Write code to create a TreeSet object. In the TreeSet's constructor, pass an object `s` to copy the entire contents of a HashSet into the TreeSet. Display the items in the TreeSet and evaluate how the result differs from a HashSet.

Exercise 8. Practice with Maps

This section will practice using Hashmaps and Treemaps. Create an ExHashMap class, where the main method creates a HashMap containing the key and value:

```
HashMap<String, Integer> myMap = new HashMap<String,  
Integer>();
```

Input the following key-value pairs into the HashMap:

```
myMap.put("Apples", (int) (Math. random () * 10));
```

```
myMap.put("Bananas", (int) (Math. random () * 10));  
myMap.put("Grapes", (int) (Math. random () * 10));  
myMap.put("Watermelons", (int) (Math. random () * 10));
```

Cherries are just as valuable as grapes:

```
myMap.put("Cherries", myMap.get("Grapes"));
```

Replace your Apple logo with a different random value:

```
myMap.put("Apples", (int) (Math. random () * 10));
```

Create a mySortedMap object:

```
TreeMap <String, Integer>mySortedMap = new TreeMap<>(myMap);  
System.out.println("my sorted map is: " + mySortedMap);
```

Run the program and print the following output:

```
My sorted map is: {Apples=8, Bananas=2, Cherries=9, Grapes=9,  
Watermelons=0}
```

Create a myMap2, change the value and key:

```
HashMap <Integer, String>myMap2 = new HashMap<>();  
for (String s : myMap.keySet())  
    myMap2.put(myMap.get(s), s);  
System.out.println("my second map is: " + myMap2);
```

Create mySortedMap2 and sort the myMap2 files by key:

```
TreeMap <Integer, String>mySortedMap2 = new TreeMap<>(myMap2);  
System.out.println("my second sorted map is: " + mySortedMap2);  
System.out.println();
```

Check myMap2 to see if 3 is the key:

```
System.out.println("my second map contains 3: " +  
myMap2.containsKey(3));
```

Check myMap2 to see if “Apples” is a value:

```
System.out.println("my second map contains Apples: " +  
myMap2.containsValue("Apples"));
```

Execute the program. Run it and check the results; the output might look like this:

```

my sorted map is: {Apples=7, Bananas=7, Cherries=6, Grapes=6,
Watermelons=6}
my second map is: {6=Cherries, 7=Bananas}
my second sorted map is: {6=Cherries, 7=Bananas}

my second map contains 3: false
my second map contains Apples: false

```

Please print a test of the dimensions of myMap and myMap2:

```

System.out.println("my map has " + myMap.size() + " keys");
System.out.println("my second map has " + myMap2.size() + "
keys");

```

Exercise (*): Explain why the size of myMap2 changes while the size of MyMap remains fixed with each different run.

Write a swap method to swap the first and second options.

QUIZZES

What are the limitations of generic programming using inheritance?	
Coercion is necessary in many cases.	
Long command code	
Overly strict type checking	
Not generic enough.	

In the naming convention for type parameters, T represents:	
Value	
Number	
Lock	
Generic type	

The keyword var is used:	
From Java version 1	
From Java version 4 onwards	

From Java version 8 onwards	
From Java version 10	

The Generic Method is the method:	
It can only be in a Generic Class.	
It can be placed in a regular layer.	
It must be a static method.	
Overridden from a generic method	

To restrict the type when declaring a generic class, use the keyword:	
Extend	
limit	
implements	
instanceof	

Initialize a List:	
Dimensions must be declared.	
The values must be initialized and cannot be changed afterward.	
Similar to initializing an object.	
The types of the elements need to be declared.	

The difference between Set and List is:	
Lists are always optimized for data storage.	
A set always sorts the elements.	
Lists allow duplicate elements, while Sets do not.	
Differences in execution speed	

Immutable lists are created by:	
new ArrayList()	
List.of()	

List.immutable()	
LinkedList	

ArrayList is faster than LinkedList when calling functions:	
get	
add	
remove	
search	

What are the characteristics of TreeSet?	
Organize the elements in a list.	
Always sort elements by value.	
Allow duplicate elements	
Sort the elements by hash value.	

What types can the variables t1 and t2 be passed to the makelist function?	
<pre> public static <T>List <T>makeList (T t1, T t2) { List <T> result = new ArrayList <T>(); result.add(t1); result.add(t2); return result ; } </pre>	
Array	
Object	
String	
List	

What is the output of the following code snippet ?	
<pre> public abstract class Component { private int x, y ; int Component (int x, int y){ </pre>	

```

        return x + y;
    }
    public static void main (String [] args) {
        Component component = new Component(3, 4);
        System.out.println(component);
    }
}

```

Print the value 7.

The program made an exception.

Print the sum of x and y.

Compilation error

What is the output of the following code snippet ?

```

class Car {
    public int gearRatio = 8 ;
    public String accelerate () {
        return "car" ;
    }
}

public class SportsCar extends Car {
    public int gearRatio = 9 ;
    public String accelerate () {
        return "accelerate" ;
    }
    public static void main (String [] args) {
        Car car = new SportsCar();
        System.out.println(car. gearRatio + " " +
        car.accelerate());
    }
}

```

Print the number: 8	
Print the text: 8 car	
Print the text: 8 accelerate	
The code snippet has compilation errors.	

What is the output of the following code snippet ? <pre>List<String> fruits = new ArrayList<String>(); fruits.add ("Apple"); fruits.add ("Banana"); fruits.add ("Strawberry"); for (String fruit : fruits) { System.out.println(fruit); if ("Apple".equals(fruit)) { fruits.remove (fruit); } }</pre>	
Remove Apple from the list.	
Remove all strings except Apple from the list.	
Remove all elements from the list.	
A ConcurrentModificationException exception occurred.	

PRACTICE Exercise S

Exercise 1. Convert a string to a number

Write a Java program to convert a list of strings into a list of integers. If a string is not an integer, skip it and continue with the other strings.

Exercise 2. Find the repeating numbers.

Given a list of numbers, show which numbers are repeated more than once in that list.

Exercise 3. Find the largest element in the SET.

Write a findMax method to find the largest element in a Set. If the Set is empty, the method should return null.

Exercise 4. Count the number of times the word appears in the sentence.

Write a Java program that uses Map to count the number of occurrences of each word in a sentence.

REFERENCES

- [1] Core Java: Fundamentals (2021), Cay Horstmann (Oracle Press Java).
- [2] Head First Java: A Brain-Friendly Guide (2022), Kathy Sierra, O'Reilly Media.
- [3] Java OOP Done Right: Create object oriented code you can be proud of with modern Java Paperback (2019), Mr Alan Mellor, Mellor Books.
- [4] Murach's Java Programming (5th Edition) (2017), Joe Murach, Mike Murach & Associates.
- [5]. Java for Absolute Beginners Learn to Program the Fundamentals the Java 9+ Way.
- [6]. Modern Java Recipes: Simple Solutions to Difficult Problems in Java 8 and 9 (2017), by Ken Kousen, O'Reilly Media.
- [7] Effective Java (2018), Joshua Bloch, Addison-Wesley Professional.